Lingo Bharat

Indic Localization Readiness Checker

Lingo.dev Hackathon 2026 Submission

Author: Kshitij | **Repo:** github.com/AjayPoshak/Lingo_Bharat

---

The Problem

India has **900 million internet users** speaking 22 official languages. Standard i18n tools solve translation but miss India-specific localization failures:

- **Numbers**: `₹1,234,567` (wrong) vs `₹12,34,567` (Indian Lakh/Crore)
- **Fonts**: Missing Devanagari/Tamil fallbacks = empty boxes
- **Formality**: Hindi "tu" register is offensive in professional UIs
- **Overflow**: Indic scripts expand 1.6-2.8x wider than English
- **Coverage**: Hindi-only reaches just 39% of Indian internet users

Translation is not localization.

The Solution

Lingo Bharat is an **ESLint for Indic Localization** built on Lingo.dev. It produces a **Bharat Readiness Score (BRS)** 0-100, graded A-F.

Check	Issue	Severity
Number Format	International vs Lakh/Crore grouping	Critical
Font Stack	Missing Indic script CSS fallbacks	Critical
Formality Register	Tu vs Aap pronoun register in Hindi	High
String Overflow	Indic expansion exceeding containers	Medium
Coverage Reach	% of India reached by supported locales	Strategic

CLI Commands

```
lingo-bharat check   - Run all checks, produce BRS
lingo-bharat fix     - Auto-fix all fixable violations
lingo-bharat serve   - Open visual dashboard
lingo-bharat report  - Export JSON report
```

Lingo.dev Integration (5 Touchpoints)

1. i18n.json Config Compatibility

Reads the same config file as Lingo.dev CLI. Zero additional configuration.

2. recognizeLocale() - Language Validation

Samples translation values and calls `LingoDotDevEngine.recognizeLocale()` to verify files contain the declared language.

```
// packages/core/src/parsers/i18n-config.ts
const engine = new LingoDotDevEngine({ apiKey });
const detected = await engine.recognizeLocale(sampleText);
```

3. localizeObject() - Batch Re-translation

`FormalityFixer` class batches all formality violations into a single `localizeObject()` call (1 API call instead of N).

```
// packages/core/src/fixer/formality-fixer.ts
const translated = await engine.localizeObject(sourceObject, {
  sourceLocale: 'en',
  targetLocale: 'hi',
});
```

4. localizeText() - Individual String Fix

Fallback for single-string re-translation.

5. CI/CD Quality Gate

GitHub Action runs `lingodotdev/lingo.dev@main` translate, then `lingo-bharat check` as quality gate. Posts BRS score table on every PR.

```
Lingo.dev Translate --> Lingo Bharat Check --> PR Comment + Artifact
```

Architecture

```
              i18n.json (Lingo.dev config)
                        |
            +---------------------+
            | buildCheckerContext  |
            |  + recognizeLocale  | <-- Lingo.dev SDK
            +---------------------+
                        |
    +--------+--------+--------+--------+
    v        v        v        v        v
  Number    Font   Formality  String  Coverage
  Format    Stack  Register  Overflow  Reach
    |        |        |        |        |
    +--------+--------+--------+--------+
                        |
```

```
              +----------------+
              | Scoring Engine |
              |  BRS 0-100     |
              +----------------+
                       |
              +--------+--------+
              v        v        v
           Terminal   JSON   Dashboard
```

Monorepo (Turborepo + pnpm)

- `packages/core` - Analysis engine, checks, parsers, scoring, fixer
- `packages/cli` - Commander.js CLI
- `packages/dashboard` - Vite + React visual report
- `apps/demo` - PayBharat demo app with seeded bugs

Demo: Before & After

Before (`pnpm reset` + `lingo-bharat check`)

```
BRS: 24/100 (Grade F)

  Number Format .......... CRITICAL  - 3 violations
  Font Stack ............ CRITICAL  - 2 violations
  Formality Register ..... HIGH      - 4 violations
  String Overflow ........ MEDIUM    - 2 violations
  Coverage ............... HIGH      - 1 violation
```

After (`lingo-bharat fix` + re-check)

```
BRS: 85/100 (Grade B)

  Number Format .......... PASS
  Font Stack ............ PASS
  Formality Register ..... PASS
  String Overflow ........ PASS
  Coverage ............... MEDIUM    - 1 recommendation
```

Test Coverage

Test File	Tests	Covers
engine.test.ts	4	BRS calculation, grades, coverage math
number-format.test.ts	1	International vs Indian detection
font-stack.test.ts	4	Missing fonts, pass, multi-locale, autoFix
formality-register.test.ts	6	Tu pronouns, verbs, aap pass, non-Hindi skip

string-overflow.test.ts	5	Tamil overflow, Hindi pass, empty keys
coverage.test.ts	7	Single/multi locale, recommendations, effort
integration.test.ts	5	Full pipeline, weighted BRS, boundaries
Total	**32**	**All 5 checks + scoring + integration**

Self-Dogfooding

Root `i18n.json` targets 5 Indic locales (hi, ta, bn, te, mr) for the CLI's own messages. Ready for `npx lingo.dev run` to translate.

Setup

```
git clone https://github.com/AjayPoshak/Lingo_Bharat.git
cd Lingo_Bharat
pnpm install && pnpm build
export LINGODOTDEV_API_KEY="lingo_sk_..."
pnpm test                                    # 32 tests pass

cd apps/demo
pnpm reset                                   # Seed bugs
npx lingo-bharat check --config ./i18n.json  # BRS ~24 (F)
npx lingo-bharat fix --config ./i18n.json    # Auto-fix
npx lingo-bharat check --config ./i18n.json  # BRS ~85 (B)
npx lingo-bharat serve --config ./i18n.json --open  # Dashboard
```

Tech Stack

Component	Technology
SDK	Lingo.dev (recognizeLocale, localizeObject, localizeText)
Language	TypeScript
CLI	Commander.js
Dashboard	Vite + React + Tailwind CSS
Testing	Vitest (32 tests, 7 files)
CI/CD	GitHub Actions + lingodotdev/lingo.dev
Monorepo	Turborepo + pnpm
Demo	Next.js (PayBharat)

Built for the Lingo.dev Hackathon 2026 "The first localization QA tool that actually understands India."